

This session will be recorded for internal use.

Deep Learning

Session 4: Computer vision (1): Convolutional neural networks

September 24, 2024

Deep learning theory

1. Deep learning in public policy
2. Deep neural networks (1)
3. Deep neural networks (2)

Applications

4. **Computer vision (1):
Convolutional neural networks**
5. Computer vision (2): Modern CNNs, implementation, and applications
6. Sequence methods and time-series analysis
7. Natural language processing (1)
8. Natural language processing (2):
Transformer models,
implementation, and applications
- Midterm exam -
9. Further topics in deep learning

Deep learning and public policy

10. Policy approaches to regulate deep learning and responsible implementation in government
11. Deep learning in practice
12. Tutorial presentations

- Images in public policy
- Basic operations that comprise the backbone of all convolutional networks
 - convolutional layers
 - pooling layers
 - multiple channels
 - structure of modern architectures
- Visualizing features

Introduction

Images and public policy

Images and public policy

- In principle rather universal language/source of information
- Images are also used by political stakeholders to communicate and frame
- Social media generates a flood of images
- Images can be very diverse
- Examples: cartoons, drone footage, portraits, camera traps, multi-spectral satellite data, scanned texts, ...
- Deep learning is a powerful tool for analyzing images



House Judiciary GOP    
@JudiciaryGOP

Protect our ducks and kittens in Ohio!



10:44 AM · Sep 9, 2024 · 70.4M Views

 4.3K

 8.1K

 33K

 1.2K



Images and public policy

Press photos

Images are political: they work with emotion, framing, they mobilize people, ...



Image: John Moore „Crying Girl on the Border“



Image: Ralph Pace, „California Sea Lion Plays with Mask“

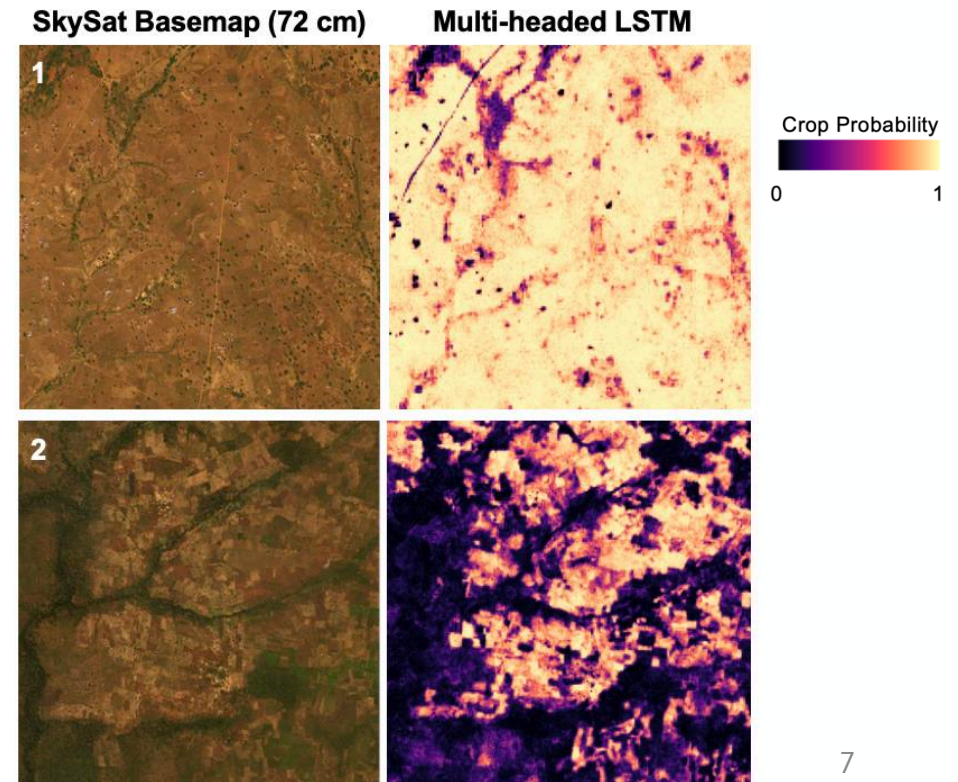
Images and public policy

Public policy research

Analysing visual frames in pictures of the migrant caravans from Central America (Torres et al., in review)



Rapid-response crop maps from satellite images in Togo (Kerner et al. 2020)



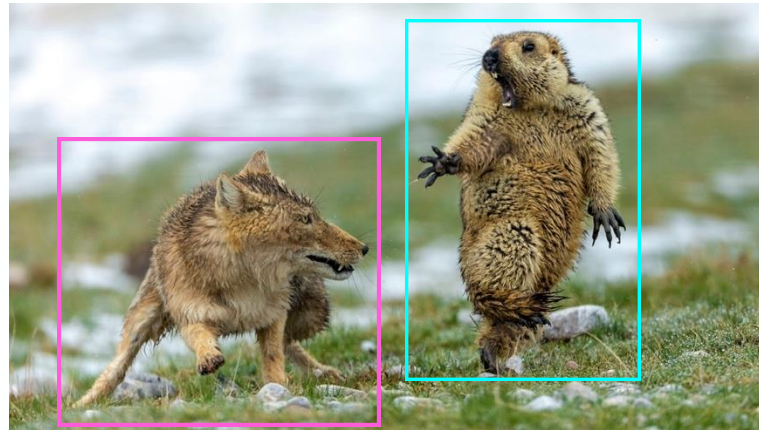
Computer vision tasks

Classification



Marmot

Object detection



Fox

Marmot

Instance segmentation



Fox

Marmot

Analyzing images

How a human sees an image

- Looks for local features
- Ignores irrelevant information



Brandenburg Gate in Berlin

Quadriga

6 columns

4 columns

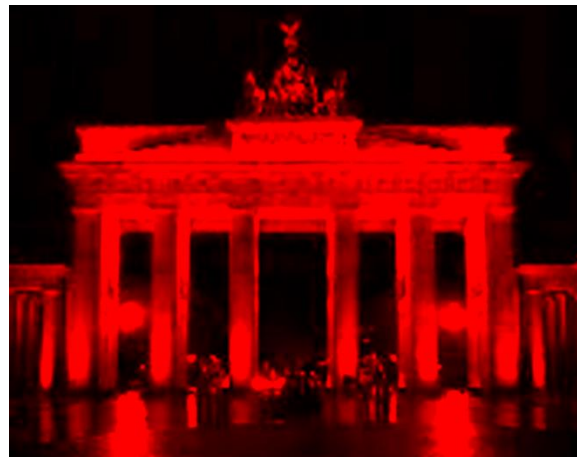


Arcade du Cinquantenaire in Brussels

Analyzing images

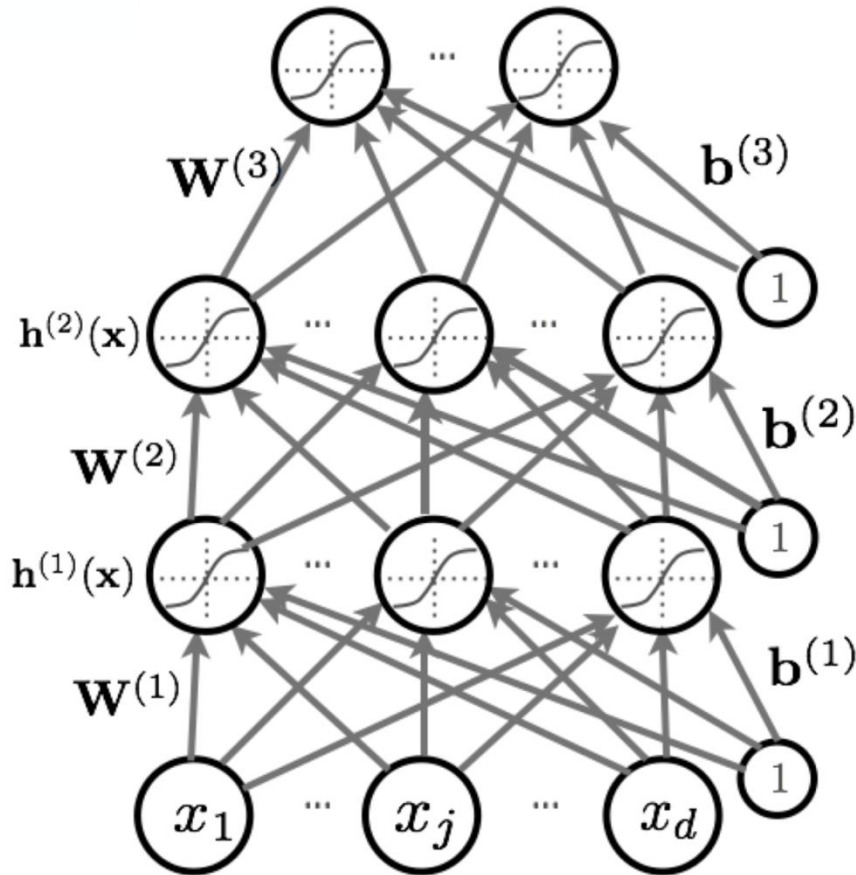
How a computer sees an image

- Matrix of pixels
- Light intensity of pixel: Numerical value (typically between 0 and 255)
- Colors: Channels
 - Typically 3 channels: RGB
 - Hyperspectral images have hundreds of channels



Convolutional layers

Fully connected layers



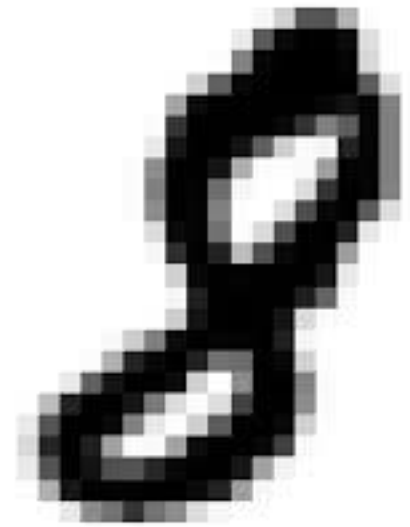
More types of layers

- So far we only worked with “fully connected” layers
- Every node is connected to every node of the next layer
- For image analysis and other applications, another type of layer has been very successful: convolutional layers

Images and input features

Spatial structure and local connectivity

- Last semester & Problem Set 1: Image as an array of pixels - the image is “flattened”
- Result: vector where each pixel is taken as one feature
- This representation is needed to feed the image as input to a fully connected neural network
- Loosing information about the spatial correlations of the pixels
- Could even permute the flattened pixels and obtain the same performance
- Problem: **Nearby pixels are most often related!**

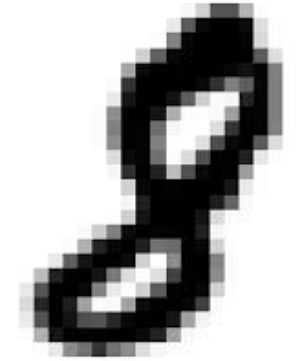


$28 \times 28 = 784$ pixels

Images and input features

High-dimensional

- Most “normal” photos are several MB in size – millions of features
- Leads to a lot of parameters in the network
- Example: Fully connected 1st hidden layer for 1MB image with 1000 hidden units would result in $10^6 \cdot 10^3 = 10^9$ parameters w_{ij} .
- Learning these many parameters both requires huge amounts of computational power but also may be tricky to optimize
- **We want high performance and less parameters!**



$28 \times 28 = 784$ pixels



1 MB picture = 10^6 pixels

Images and input features



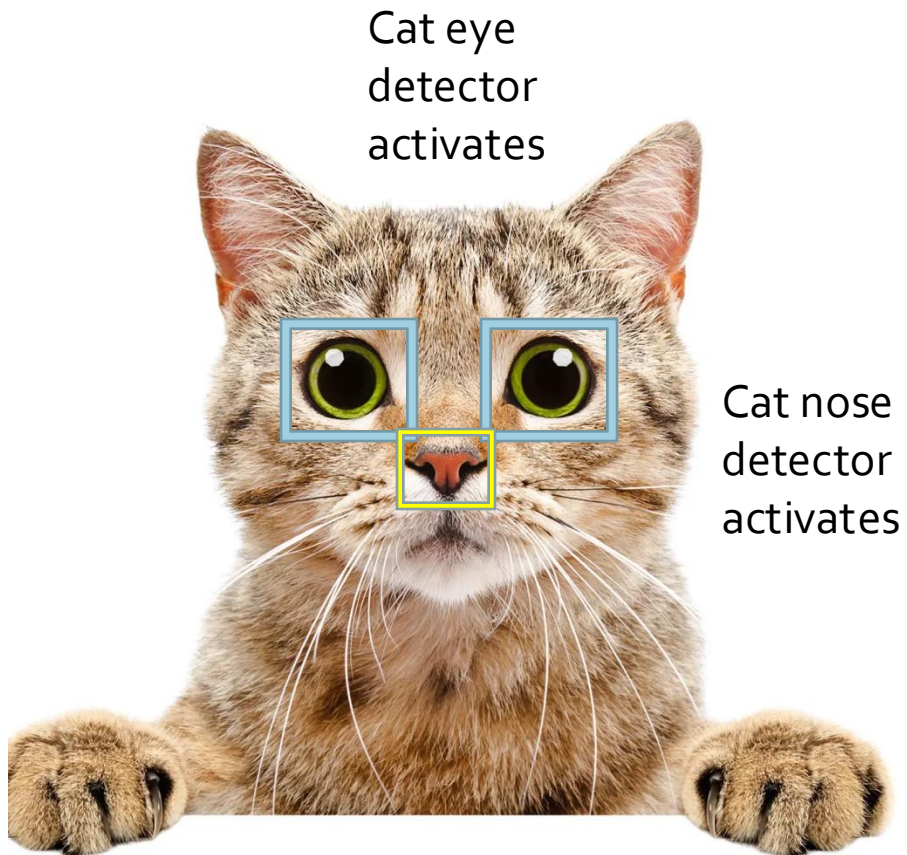
Image: RBB

Translation invariance and equivariance

Task: Are there helmets in this image?

- It matters what helmets look like, but it doesn't matter where the helmets are located
- Helmets look very similar
- Would want to sweep with a helmet detector to see the matches
- **We need a new type of layer to work as such a "detector"**

Images and input features



Translation invariance and equivariance

Task: Image classification

- Generalized and translation invariant features are very relevant for recognizing the content of images more generally
- Detectors/filters recognize characteristic features

Convolutional neural network (CNN)

Properties of CNNs

- Convolutional neural networks (CNNs) contain **convolutional layers** that leverage dependencies of nearby features (**local connectivity**)
- Can deal with **translation invariance and equivariance**
- Earlier layers focus on local regions, later layers focus on the whole image (**locality**)
- Can also deal with invariance in illumination
- Widely used in computer vision
- Also, competitive in exploiting one-dimensional sequence structure in time series analysis, audio and text.

CNNs tend to be computationally efficient:

- require fewer parameters than fully connected architectures (**sharing parameters** exploiting that spatial correlation is local)
- convolutions are easy to parallelize across GPU cores.

Convolutional layer

Discrete convolution

- Detector or filter is called **kernel**
- Grid/matrix of learned features
- For image input $X \in \mathbb{R}^{d_x \times d_y}$ (matrix with rows and columns of pixels) and kernel $K \in \mathbb{R}^{r \times r}$ (matrix of weights) the discrete convolution is computed as

$$(X * K)_{ij} = \sum_p \sum_q x_{i+p, j+q} k_{r-p, r-q}$$

- The offsets p (rows) and q (columns) run from $0, \dots, r - 1$
- Example:

$$\begin{array}{|c|c|c|} \hline 0 & 80 & 40 \\ \hline 20 & 40 & 0 \\ \hline 0 & 0 & 40 \\ \hline \end{array} * \begin{array}{|c|c|} \hline 0 & 0,25 \\ \hline 0,5 & 1 \\ \hline \end{array} =$$

$x \qquad k$

$r = 2,$
 p and q run
 from 0 to 1

Convolutional layer

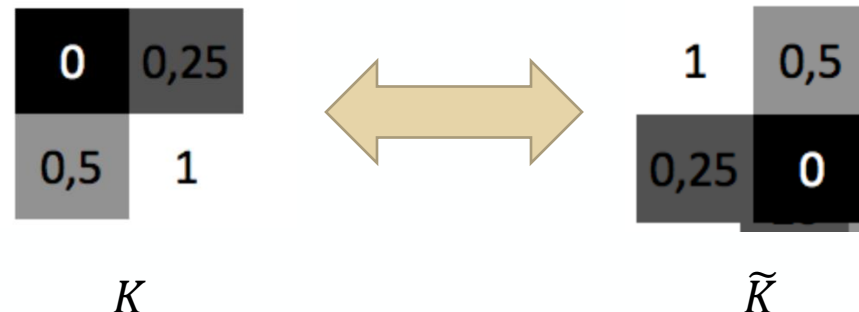
Example: Discrete convolution

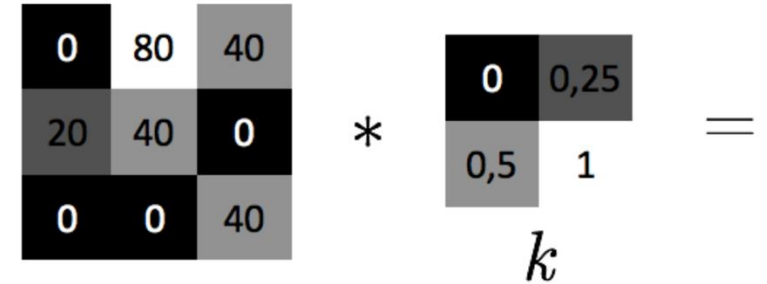
We want to compute

$$(X * K)_{ij} = \sum_p \sum_q x_{i+p, j+q} k_{r-p, r-q}$$

with kernel size $r = 2$. The offsets p (rows) and q (columns) run from 0 to 1.

Notice the minus sign in the indices of k : We can think of this as a filter $\tilde{K}$ that is K with rows and columns flipped, and then the operations simplifies considerably.





0	80	40
20	40	0
0	0	40

x

*

0	0,25
0,5	1

k

=

Convolutional layer

Example: Discrete convolution

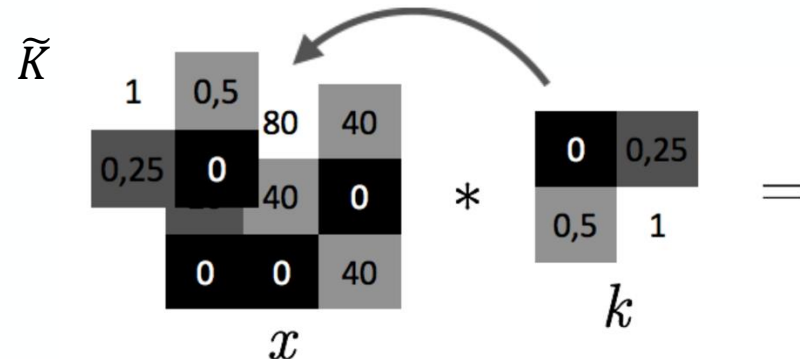
We want to compute

$$(X * K)_{ij} = \sum_p \sum_q x_{i+p, j+q} k_{r-p, r-q}$$

with kernel size $r = 2$. The offsets p (rows) and q (columns) run from 0 to 1.

Now, we can use $\tilde{K}$ to more conveniently run over the offsets and multiply each element of $\tilde{K}$ with the corresponding element of the upper left 4 by 4 matrix of X , and then sum:

$$1 \cdot 0 + 0.5 \cdot 80 + 0.25 \cdot 20 + 0 \cdot 40 = 45$$



$$\tilde{K} * k =$$

Convolutional layer

Example: Discrete convolution

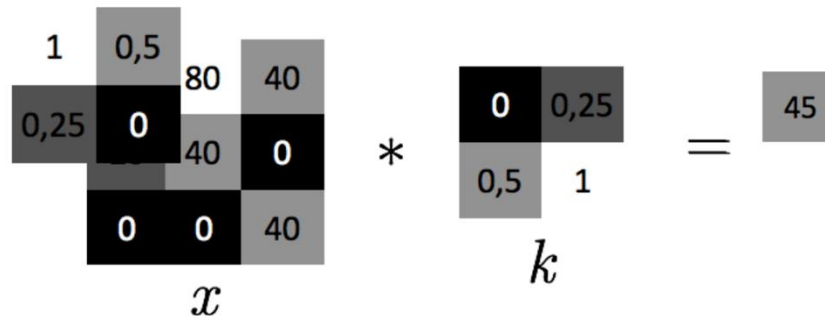
We want to compute

$$(X * K)_{ij} = \sum_p \sum_q x_{i+p, j+q} k_{r-p, r-q}$$

with kernel size $r = 2$. The offsets p (rows) and q (columns) run from 0 to 1.

Now, we can use $\tilde{K}$ to more conveniently run over the offsets and multiply each element of $\tilde{K}$ with the corresponding element of the upper left 4 by 4 matrix of X , and then sum:

$$1 \cdot 0 + 0.5 \cdot 80 + 0.25 \cdot 20 + 0 \cdot 40 = 45$$



$$\begin{array}{|c|c|c|} \hline 0 & 80 & 40 \\ \hline 20 & 40 & 0 \\ \hline 0 & 0 & 40 \\ \hline \end{array} \quad * \quad \begin{array}{|c|c|} \hline 0 & 0,25 \\ \hline 0,5 & 1 \\ \hline \end{array} = 45$$

x k

Convolutional layer

Using convolutional kernels as weights

- For the computer, the order of the weights doesn't matter as long it is consistent.
- We can define the kernel with rows and columns flipped as our weight matrix $\tilde{K} = W$.
- This is also called the discrete (cross-)correlation of X and W .
- The name Convolutional Neural Network comes from the mathematical concept of convolutions, networks directly implement the correlation.
- The output of the operation convolution/correlation forms the activation of our hidden convolutional layer.
- We then apply a non-linear activation function to the output
- E.g., we apply a sigmoid to each entry of the resulting activation matrix.

Excercise

Compute the following convolution (not cross-correlation)

0	1	0
0	1	0
0	0	0

 $*$

0	1
0	1

 $=$

Effect of applying a convolution

Illustrative example



0	0	255	0	0
0	0	255	0	0
0	0	255	0	0
0	255	0	0	0
255	0	0	0	0

*

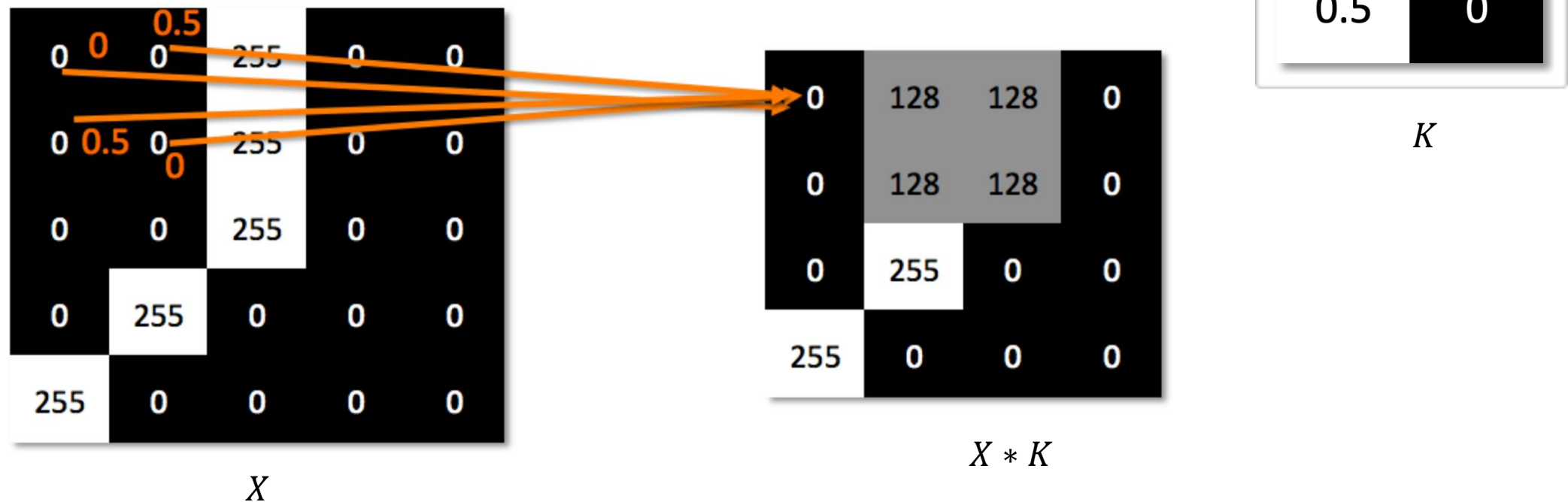
0	0.5
0.5	0

=

In this example, the kernel is the same as the kernel/weights with the rows and columns flipped

Effect of applying a convolution

Illustrative example

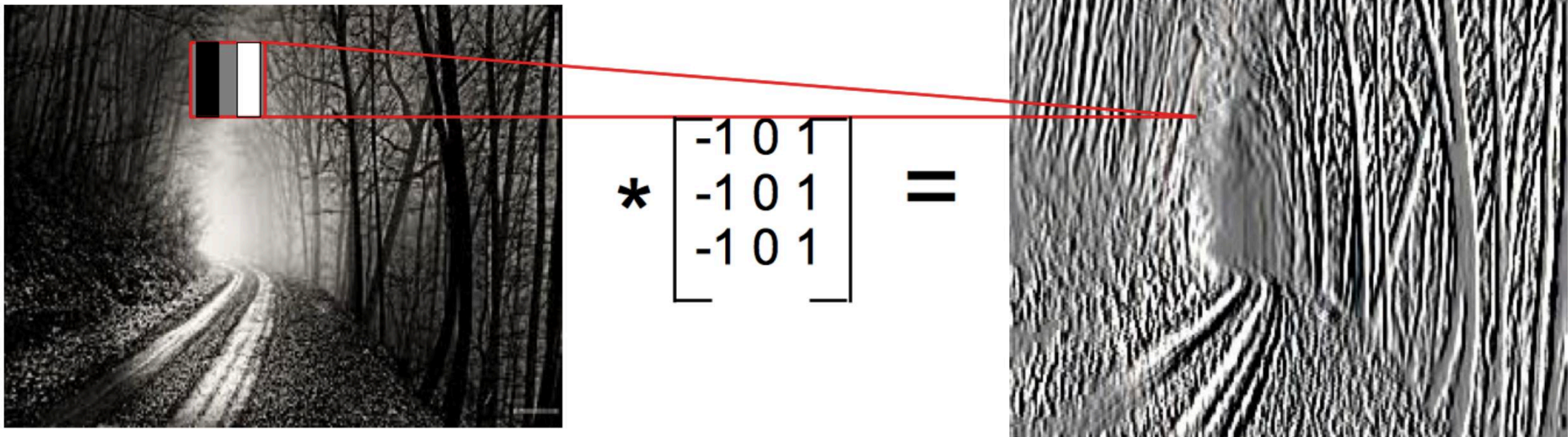


- The convolution highlights the areas that are similar to the filter.
- If we pass this (rescaled) through a sigmoid activation function, we would get the neurons that highlight these areas.

Effect of applying a convolution

Change from dark to light

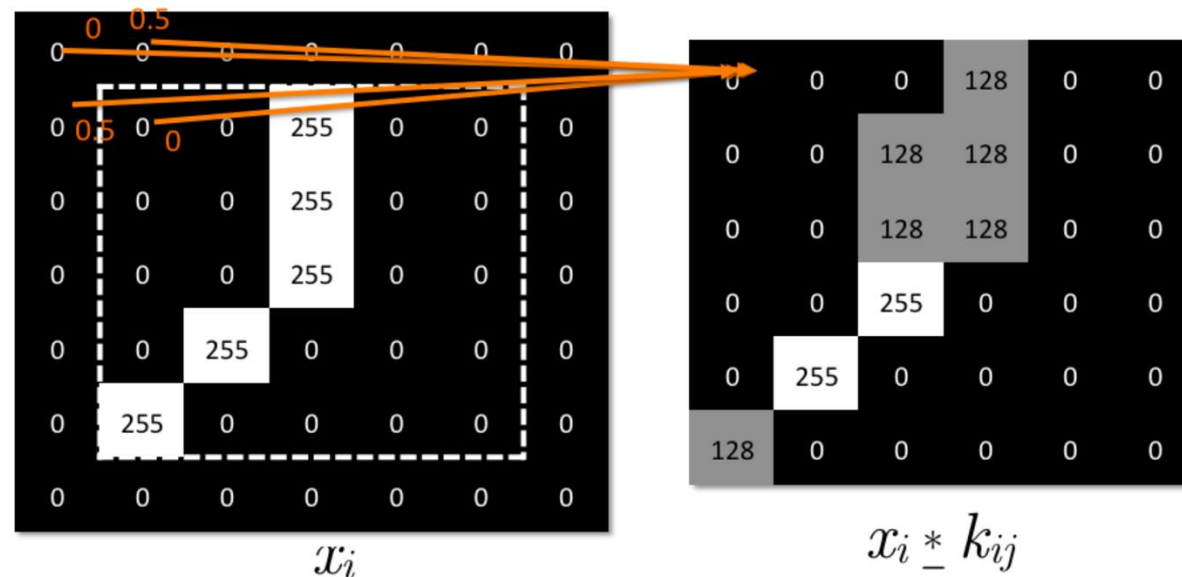
(This case shows a correlation operation)



Border of the input image

Loosing information at the edges of the input

- Ignore them $\Rightarrow$ image gets smaller
- Adding pixels
 - Suppose the image continuous with the last pixels
 - Mirror the pixels on the border
 - Suppose they are zeros (“**zero-padding**”)



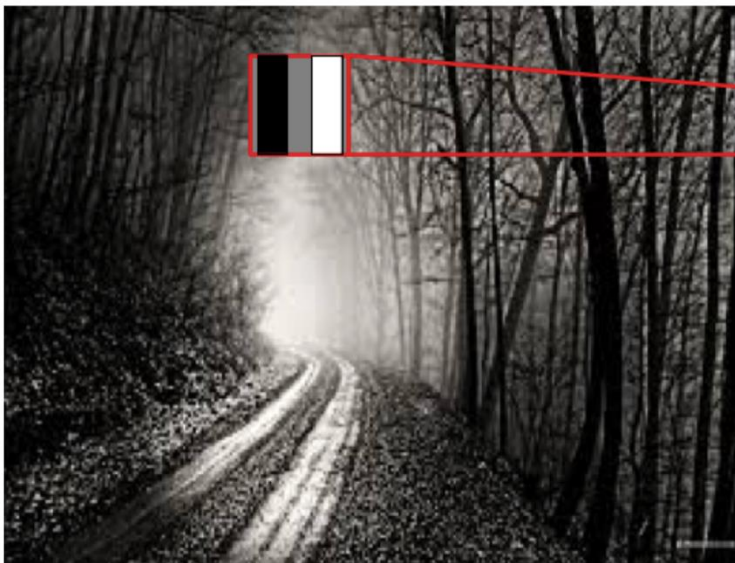
Pooling layers

Motivation

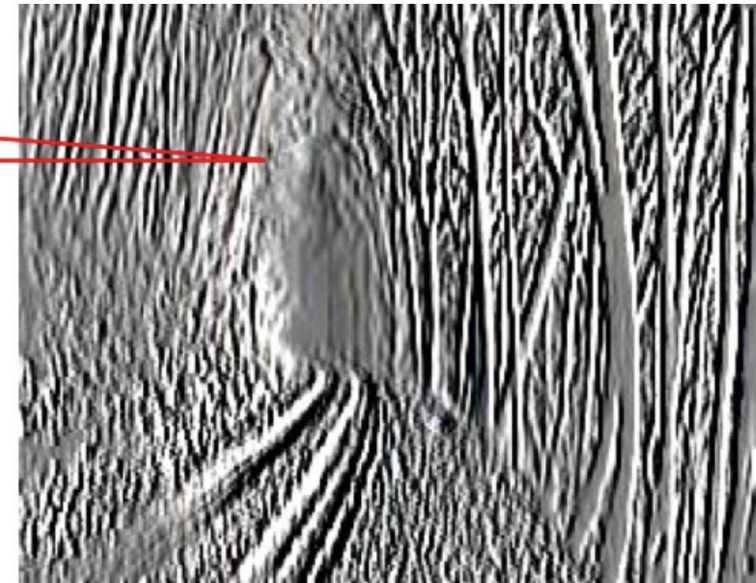
From local to global

- Now we have information for (almost) every pixel about where the light gradients are (a lot of points in the feature map)
- We want to know more aggregate information, e.g. classification if there is a street in the image
- May not care about local translations
- Need to “pool” the information!

Example convolutional layer



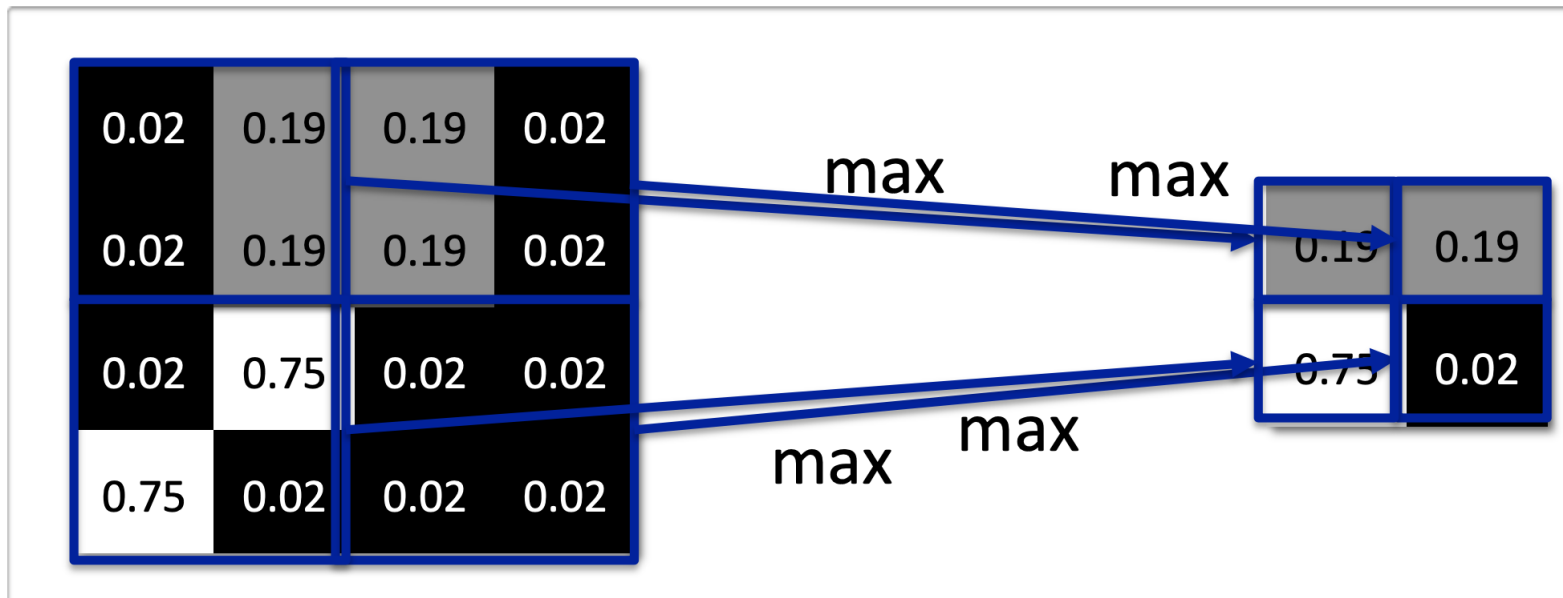
$$\begin{matrix} * & \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} & = \end{matrix}$$



Pooling operation

Example max pooling (with stride = 2)

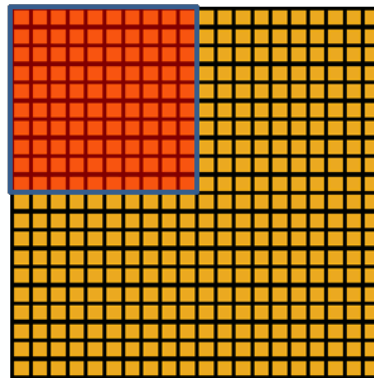
- We take the maximum output of each 4 features in the feature map to form a new **pooling layer**
- Reduces the dimensionality of the feature map considerably
- Keeps the information about what the largest activations are



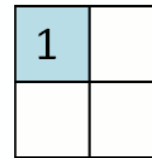
Pooling operation

Example max pooling

- We take the maximum output of each 100 features in the feature map to form a new **pooling layer**
- Reduces the dimensionality of the feature map considerably
- Keeps the information about what the largest activations are



Convolved
feature

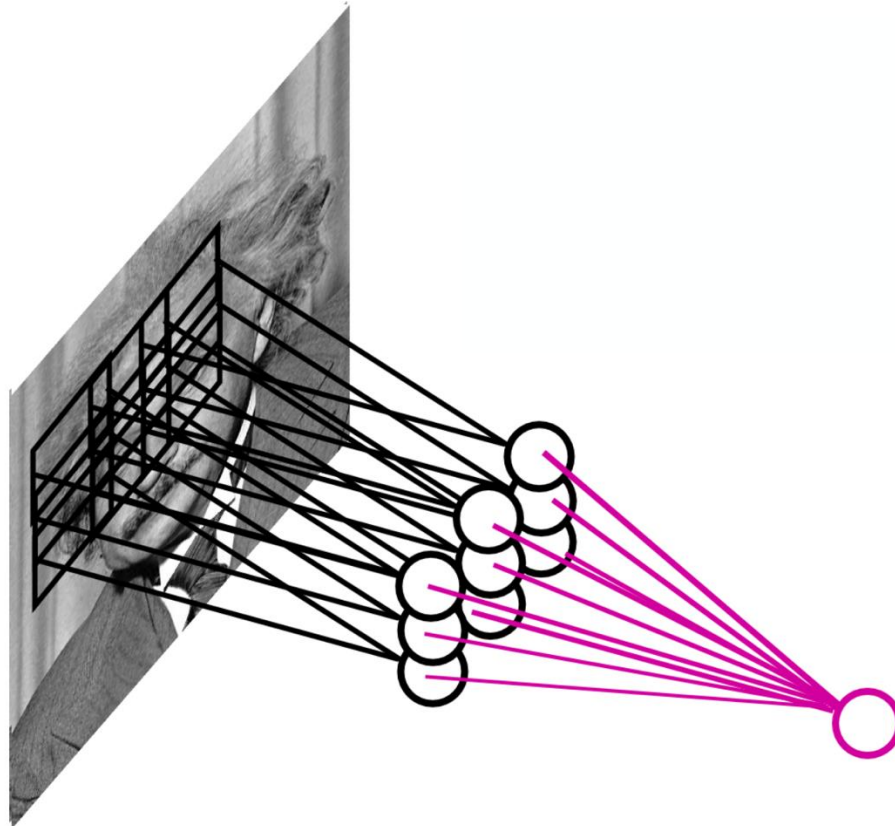


Pooled
feature

Local translation invariance

Abstract example

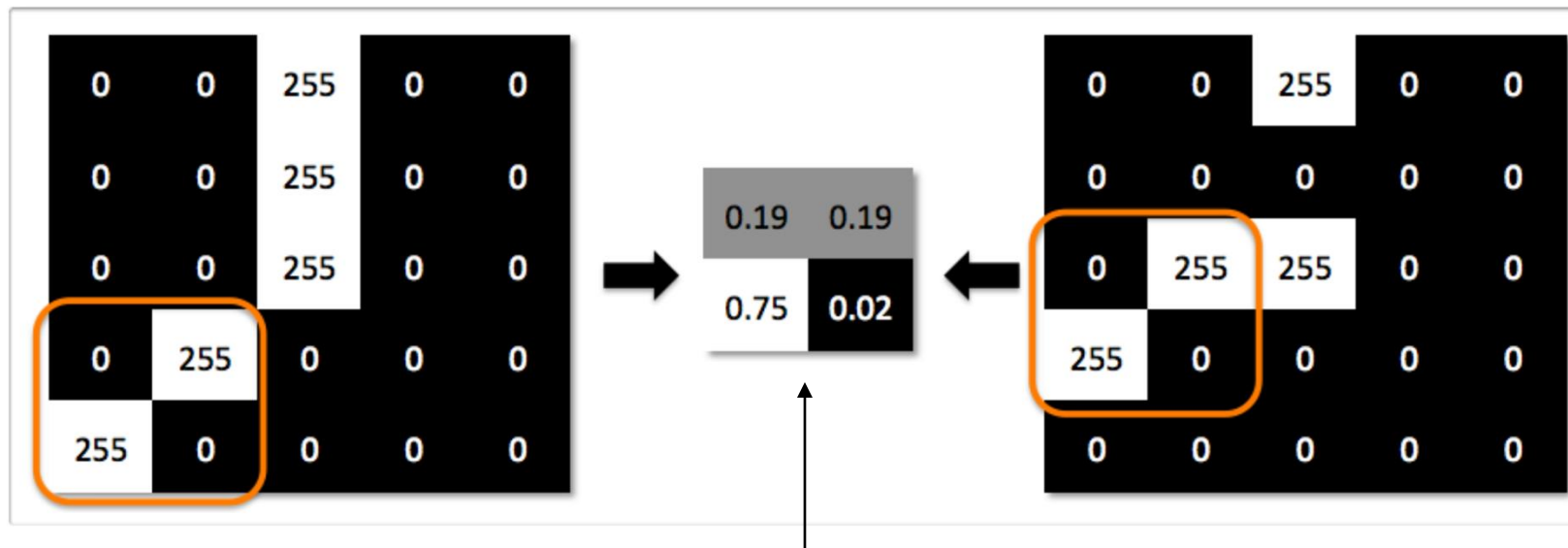
Making the detection robust to the exact location of the eye



Local translation invariance

Illustrative example

- Both of these input data are resulting in the same pooling layer output
- Most often desirable since we want robustness to the exact spatial location of features



After convolution, sigmoid
activation and pooling

Pooling

Overview and mathematical formulation

- Pooling (and subsampling) is a deterministic operation on local, (most often non-overlapping) neighborhoods of hidden units
- Overlapping depends on the **stride** (number of pixels the pooling region is moved)
- Pooling introduces invariance to local translations
- Reduces the number of hidden units passed on to the following hidden layers
- Max pooling

$$h_{jk}^{[l]} = \max_{p,q} h_{j+p,k+q}^{[l-1]}$$

- Average pooling

$$h_{jk}^{[l]} = \frac{1}{m^2} \sum_{p,q} h_{j+p,k+q}^{[l-1]}$$

where p and q are the vertical and horizontal indices in the local neighborhoods respectively (run from 0 to size of pooling region $m - 1$).

Convolutional neural network

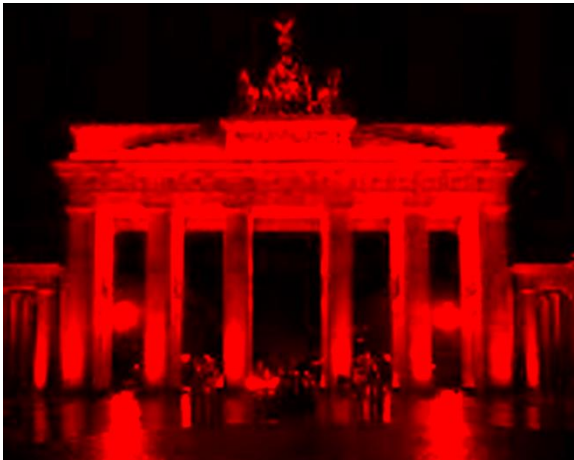


Multiple input channels

Color images

- Colors are represented by channels
- Often 3 channels (RGB), sometimes more channels
- 1 input image now is a $d_x \times d_y \times 3$ tensor:

Channel 1 (red/R)



Channel 2 (green/G)



Channel 3 (blue/B)

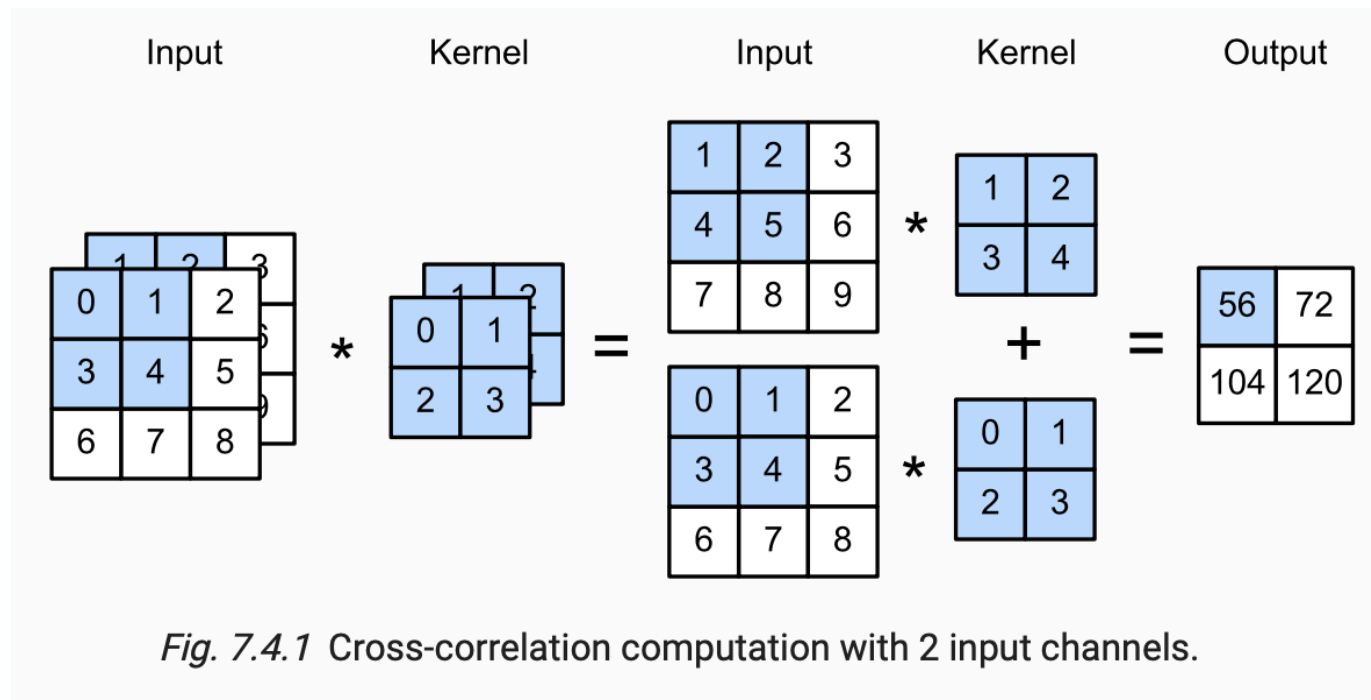


Multiple input channels

Operation with multiple channels

We have a kernel for each channel and add the results together for the output.

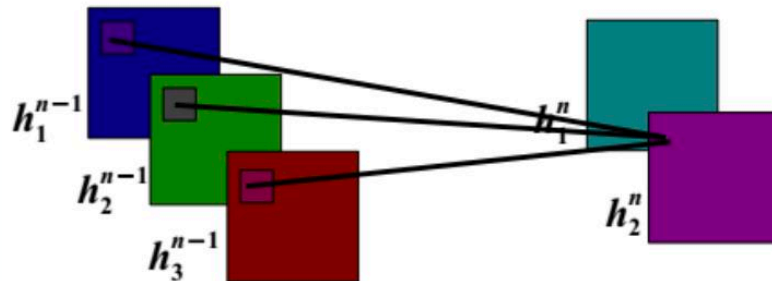
$$(1 \cdot 1 + 2 \cdot 2 + 4 \cdot 3 + 5 \cdot 4) + (0 \cdot 0 + 1 \cdot 1 + 3 \cdot 2 + 4 \cdot 3) = 56$$



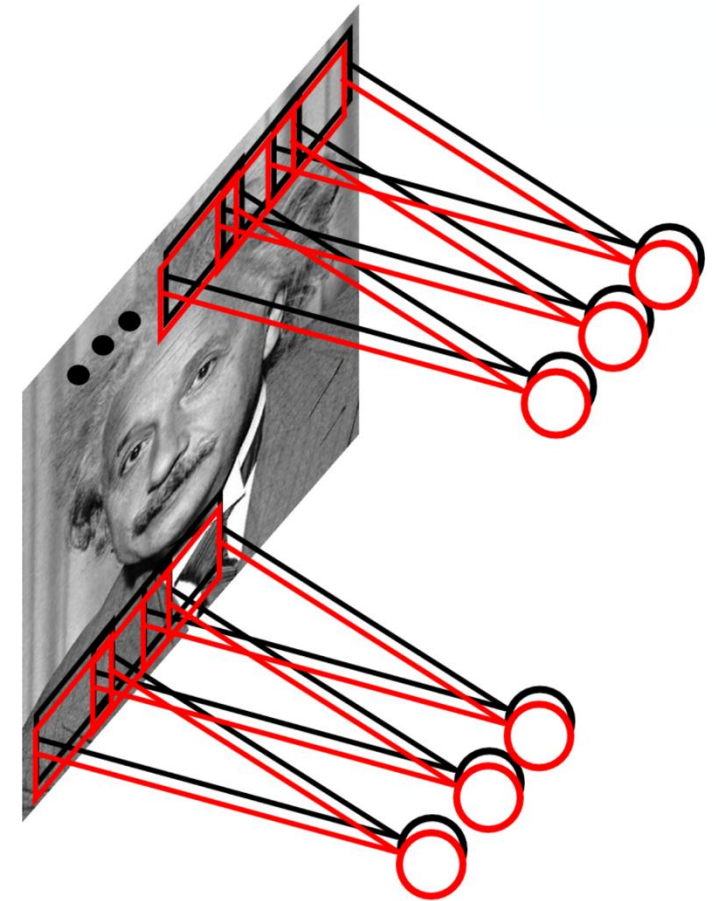
Multiple output channels/feature maps

Using many filters

- We often want to run multiple filters that pick up different features from our images
- Each filter creates a new **feature map** (many layers in parallel)
- For multiple input channels, we have a filter for each channel and output feature map. We sum over the filters for the input channels



Three input and two output channels

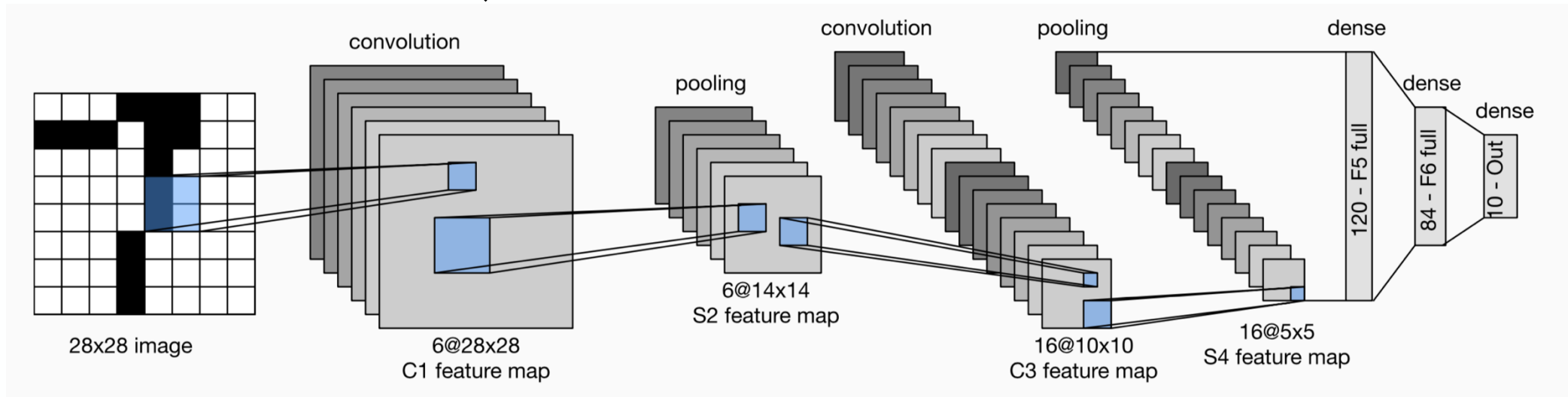


Two output channels

Convolutional neural network

Example LeNet

- Convolutional encoder (left part) and dense block (=fully connected) (right)
- Alternating convolutional layers and pooling layers
- 6 kernels of size 2×2 giving rise to 6 feature maps in Layer C1
- Layer C1 is tensor of dimension $6 \times 28 \times 28$
- Convolutional blocks are flattened when fed into the dense layers (3 dimensional $\rightarrow$ 1 dimensional)



Training a CNN

Notes on training

- For classification, the output layer is a regular, fully connected layer with softmax function
- CNNs are trained with (mini-batch) stochastic gradient descent
- Backpropagation is used similarly as in the fully-connected network
- Gradients are also passed through the convolution operation and pooling operation
- For example max pooling: The max/"winning" hidden units get the gradients, the rest is zero



Visualizing features

Feature visualization

What does a CNN learn? GoogleLeNet

Examples of learned features (tiling of neurons)

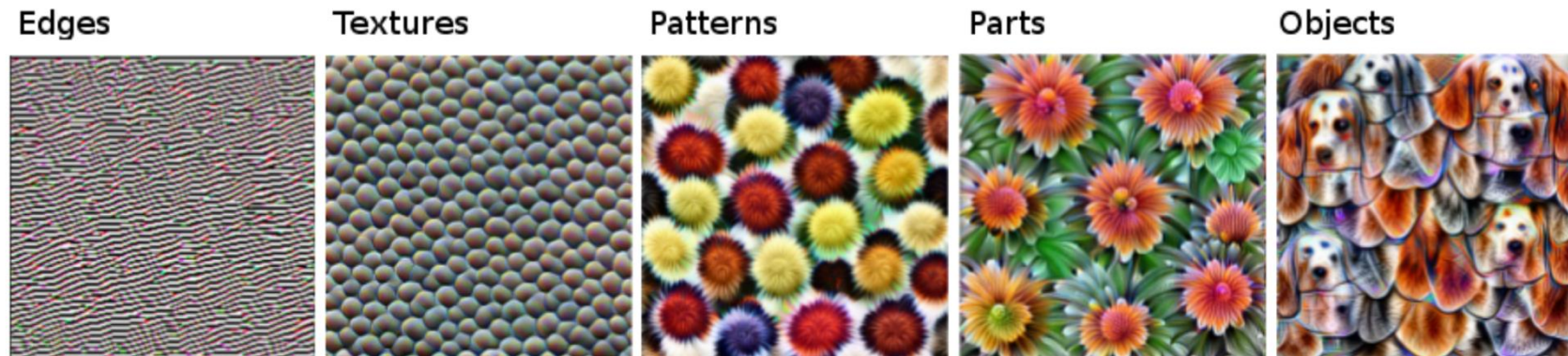


FIGURE 10.1: Features learned by a convolutional neural network (Inception V1) trained on the ImageNet data. The features range from simple features in the lower convolutional layers (left) to more abstract features in the higher convolutional layers (right). Figure from Olah, et al. (2017, CC-BY 4.0) <https://distill.pub/2017/feature-visualization/appendix/>.

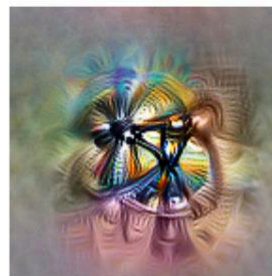
Feature visualizations

How images were created

Authors looked searched for images that caused a neuron or channel to activate

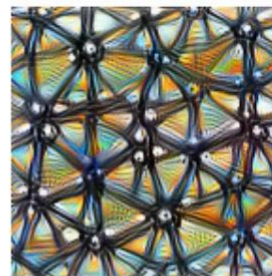
Different **optimization objectives** show what different parts of a network are looking for.

n layer index
x,y spatial position
z channel index
k class index



Neuron

$\text{layer}_n[x, y, z]$



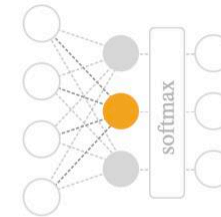
Channel

$\text{layer}_n[:, :, z]$



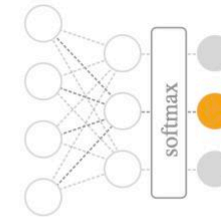
Layer/DeepDream

$\text{layer}_n[:, :, :]^2$



Class Logits

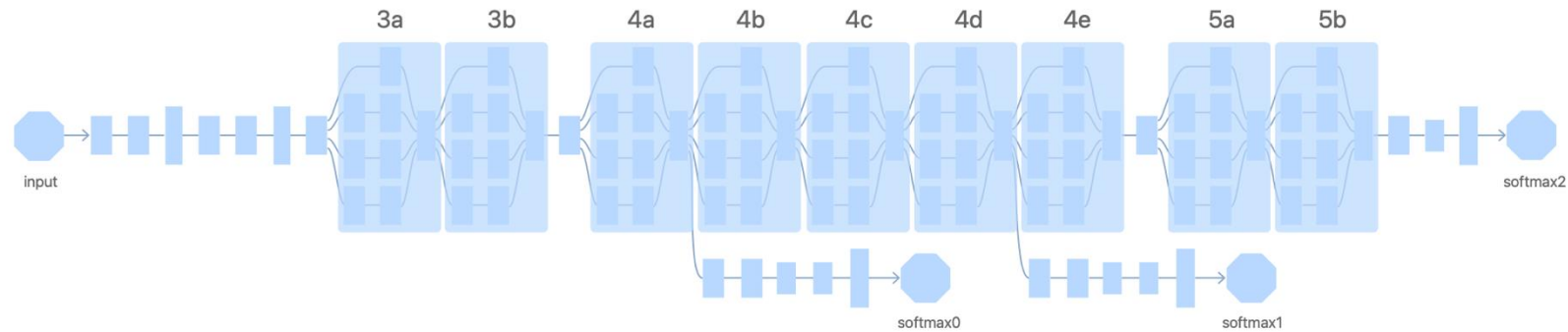
$\text{pre_softmax}[k]$



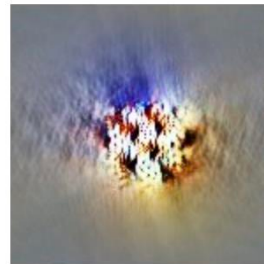
Class Probability

$\text{softmax}[k]$

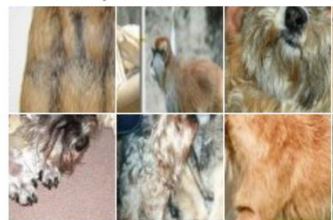
Feature visualization



LAYER 3A, UNIT 56



Neuron Objective



Dataset examples

LAYER 4C, UNIT 9



Neuron Objective



Dataset examples

LAYER 5B, UNIT 32



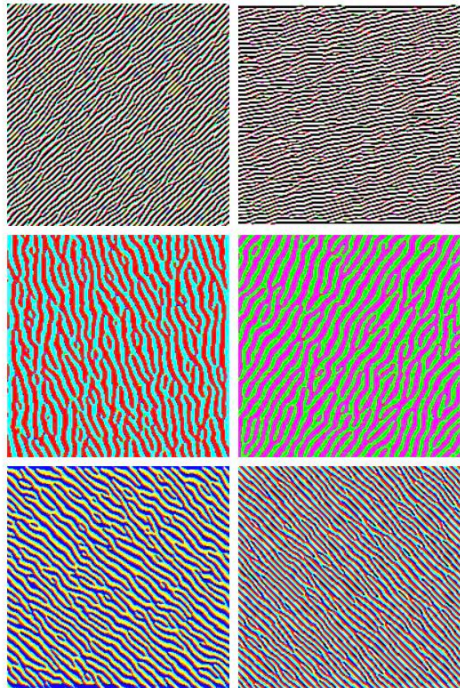
Neuron Objective



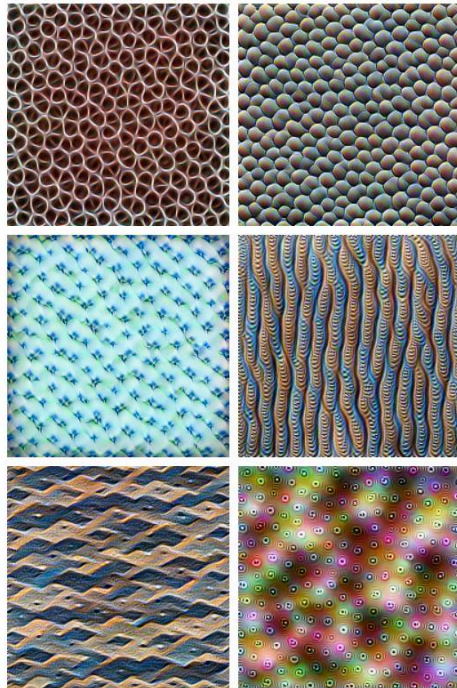
Dataset examples

Feature visualizations

From textures to objects



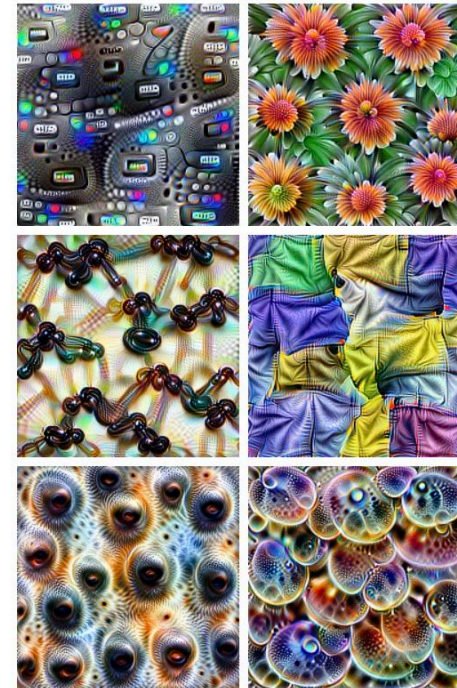
Edges (layer conv2d0)



Textures (layer mixed3a)



Patterns (layer mixed4a)



Parts (layers mixed4b & mixed4c)



Objects (layers mixed4d & mixed4e)

Feature visualization allows us to see how GoogLeNet [1], trained on the ImageNet [2] dataset, builds up its understanding of images over many layers. Visualizations of all channels are available in the [appendix](#).

- Images in public policy
- Basic operations that comprise the backbone of all convolutional networks
 - convolutional layers
 - pooling layers
 - multiple channels
 - structure of modern architectures
- Visualizing features

Hertie School
Friedrichstraße 180
10117 Berlin, Germany
T +49 (0)30 259219-0
F +49 (0)30 259219-11
info@hertie-school.org
www.hertie-school.org